ORACLE AUTONOMOUS DATABASE SHARED

OCI IAM DB Token Federation

Human & M2M Authentication — Empirical Validation Matrix

Python (python-oracledb 3.4.2) | Java (ojdbc17 23ai) | SQL Developer Classic | SQL Developer VSCode | SQL*Plus | OCI CLI

THICK (Instant Client 19.29 / 23.26) | THIN (pure Python / pure Java) | OCI_INTERACTIVE | M2M | Human SSO

Private Endpoint | mTLS | Corporate NTLM Proxy | 4 Oracle Support SRs | 3 Bugs / ERs confirmed

Alessandro Moccia
Oracle Solution Architect | OCI Specialist | Oracle ACE Program
March 2026
github.com/alexmocciaoci/oracle-adb-iam-token-federation

1. Abstract

This paper presents a structured empirical validation of OCI IAM DB Token authentication against Oracle Autonomous Database Shared (ADB-S), covering 23 controlled test scenarios across Python (python-oracledb 3.4.2), Java (ojdbc17 23ai), SQL Developer Classic, SQL Developer VSCode Extension, SQL*Plus, and OCI CLI.

The validation isolates driver-layer protocol behaviour, token acquisition mechanisms, proxy traversal constraints, and Oracle Net connect negotiation characteristics under both Human (interactive OAuth2 SSO browser) and M2M (machine-to-machine) authentication flows, across two distinct versions of Oracle Instant Client (19.29 and 23.26) and corporate NTLM proxy topologies.

Five failure patterns are root-caused with engineering evidence. Four Oracle Support SRs were opened, two Enhancement Request Bugs and one Code Bug were acknowledged by Oracle Engineering. The result is a deployment-grade pattern selection framework for enterprise environments.

2. IAM DB Token — Architecture and Authentication Model

2.1 Two-Phase Authentication Model

OCI IAM DB Token authentication separates the end-to-end flow into two completely independent phases with distinct network paths:

- **Phase A — Token Acquisition (IAM plane) —** OCI IAM issues a short-lived JWT db-token (1-hour TTL). The client authenticates to OCI IAM via API Key, Instance Principal, Resource Principal, or interactive browser OAuth2 (Human SSO). The token and its associated ephemeral private key (oci_db_key.pem) are returned. This phase requires HTTPS connectivity to OCI IAM public endpoints — it must traverse any corporate proxy.
- **Phase B — Token Presentation (Database plane) —** The JDBC/Python driver presents the token + PoP signature during the Oracle Net connection handshake. ADB-S validates the JWT signature via OCI IAM JWKS and validates the Proof-of-Possession signature. This phase connects to the ADB-S Private Endpoint via the OCI VCN — no proxy traversal required.

The separation of these two phases is the architectural key to M2M in corporate proxy environments: token acquisition traverses the proxy; database connection bypasses it entirely.

2.2 Proof-of-Possession (PoP) — Structural Replay Protection

Token acquisition generates two artefacts: (1) a signed JWT containing the IAM principal identity and an embedded public key, and (2) an ephemeral RSA private key (oci_db_key.pem). During the Oracle Net handshake, the driver signs a server challenge with the private key. ADB-S validates both the JWT signature and the PoP signature. Possession of the token file alone is structurally insufficient for authentication — this is not a configurable option, it is enforced by the protocol.

2.3 Three Client Stacks — Three Different Behaviours

The behaviour of IAM DB Token authentication is fundamentally determined by which client stack handles the Oracle Net connect negotiation. This is the single most important architectural factor in this validation:

- **THICK (libclntsh / oci.dll — Oracle Instant Client) —** The OCI native C library handles wallet loading, TOKEN_AUTH parameter parsing, OCI_INTERACTIVE browser launch, PoP signing, and TLS natively. Configured via TOKEN_AUTH in sqlnet.ora or the connect descriptor (tnsnames.ora). Used by SQL*Plus, python-oracledb THICK mode, JDBC THICK (jdbc:oracle:oci:), SQL Developer THICK.
- **THIN (pure Python — python-oracledb 3.4.2) —** The driver implements the full Oracle Net protocol in pure Python without Oracle Instant Client. IAM token is injected via the access_token parameter of ConnectParams() as a 2-tuple (token_string,

private_key_string) or as a callable. Cannot use TOKEN_AUTH=OCI_TOKEN in connect descriptor syntax (documented in Appendix B). The access_token injection path — documented as the supported alternative — also fails on ADB-S Private Endpoint (see Finding 1).

- **JDBC THIN (ojdbc17 23ai — pure Java) —** Java driver with pure Java Oracle Net implementation. Token injected via CONNECTION_PROPERTY_TOKEN_AUTHENTICATION + CONNECTION_PROPERTY_TOKEN_LOCATION. Seamless OCI_INTERACTIVE Human SSO available via ojdbc-extensions (ojdbc-provider-oci module) which wraps the OCI Java SDK to perform the browser OAuth2 redirect.

2.4 OCI_INTERACTIVE — Two Independent Implementations

There are two completely independent implementations of the OCI_INTERACTIVE Human SSO flow, with different failure modes:

- **THICK native (libclntsh / oci.dll) —** TOKEN_AUTH=OCI_INTERACTIVE in sqlnet.ora or tnsnames.ora instructs the OCI native library to launch the system browser and acquire the db-token directly — no Java SDK, no ojdbc-extensions. Feature introduced in Instant Client 23.x. Tested with SR on request per IC 19.x and 23.x. Both fail (see Finding 2).
- **JDBC THIN via ojdbc-extensions (ojdbc-provider-oci) —** oracle.jdbc.tokenAuthentication=OCI_INTERACTIVE instructs ojdbc17 to delegate token acquisition to the ojdbc-provider-oci module, which calls OCI IAM via the OCI Java SDK (GenerateScopedAccessToken), launches the browser, returns the token. Requires ojdbc-extensions library explicitly added to classpath. Works without proxy (T18 SUCCESS). Fails under NTLM proxy (T14/T15/T17 ER/FAILURE).

3. Test Environment

Component	Details
OCI Region	eu-milan-1
ADB-S	Autonomous Database Shared — private OCI tenancy
Endpoint	Private Endpoint — no public internet exposure, VCN only
mTLS	Mandatory — cwallet.sso (THICK) + ewallet.pem (THIN) + tnsnames.ora
IAM config	DBMS_CLOUD_ADMIN.ENABLE_EXTERNAL_AUTHENTICATION (type => 'OCI_IAM')
Python	Python 3.12 + python-oracledb 3.4.2 + OCI Python SDK 2.x
Instant Client	19.29.0.0.0 and 23.26.1.0.0 — tested separately on SR Engineering request
Java	JDK 21 + ojdbc17 23.26.1.0.0 + UCP17 + OCI Java SDK 3.x + ojdbc-extensions 1.0.x
SQL Developer	Classic 24.3.1 + VSCode Extension 25.3.2
SQL*Plus	Bundled with Instant Client 19.29 and 23.26
OCI CLI	OCI CLI 3.4+ — oci iam db-token get
Network	Corporate NTLM Proxy + OCI Private Endpoint via VPN
Client OS	Windows 11 — corporate enterprise environment

4. Complete Validation Matrix — 23 Test Scenarios

The table below summarises all 23 test scenarios. SR numbers and Bug IDs are anonymised (available in the accompanying Excel file). Test IDs T1–T23 map directly to the Excel rows including exact error text, SR status, and engineering classification.

ID	Stack	Mode	Token Source	Proxy	Actor	Result	Key Note / SR / Bug
T1	Python oracledb	THICK 23.x	OCI CLI file + access_token	No	M2M	**SUCCESS**	Validated baseline — THICK + file token
T2	Python oracledb	THICK 23.x	OCI SDK oci_tokens plugin	No	M2M	**SUCCESS**	In-process SDK plugin — no CLI subprocess
T3	Python oracledb 3.4.2	THIN	OCI CLI → access_token tuple	No	M2M	**FAILURE**	SR open — DPY-6005/DPY-6000 — connect negotiation
T4	Python oracledb 3.4.2	THIN	OCI SDK → access_token callable	No	M2M	**FAILURE**	SR open — same failure, same phase as T3
T5	SQL Dev Classic	JDBC THIN	OCI CLI file TOKEN_AUTH=OCI_TOKEN	No	M2M	**SUCCESS**	Reference — JDBC file token
T6	SQL Dev Classic	THICK OCI 23x	OCI CLI TOKEN_AUTH=OCI_TOKEN	No	M2M	**SUCCESS**	Reference — THICK file token via sqlnet.ora
T7	SQL Dev VSCode	JDBC THIN	ojdbc-provider-oci OCI_INTERACTIVE	NTLM	Human	**BUG**	Bug confirmed — provider not implemented in extension
T8	SQL Dev Classic	THICK IC 23.26	TOKEN_AUTH=OCI_INTERACTIVE	No	Human	**FAILURE**	ORA-25714 — browser never launched — SR open
T9	SQL Dev Classic	THICK IC 19.29	TOKEN_AUTH=OCI_INTERACTIVE	No	Human	**FAILURE**	ORA-01017 — feature absent in 19.x
T10	SQL*Plus	THICK IC 23.26	TOKEN_AUTH=OCI_INTERACTIVE	No	Human	**FAILURE**	ORA-25714 — confirms bug in libclntsh/oci.dll
T11	SQL*Plus	THICK IC 19.29	TOKEN_AUTH=OCI_INTERACTIVE	No	Human	**FAILURE**	ORA-01017 — feature absent in 19.x
T12	OCI CLI	Native CLI	OCI CLI browser SSO	Proxy	Human	**SUCCESS**	IAM reachable — browser flow works via CLI

T13	Any client	Any	Wallet TLS only	No	—	**SUCCESS**	Connectivity + mTLS baseline
T14	SQL Dev Classic	JDBC THIN	ojdbc-provider-oci OCI_INTER.	NTLM	Human	**ER**	ER filed — HTTP 407 — NTLM proxy blocks JVM
T15	SQL Dev VSCode	JDBC THIN	ojdbc-provider-oci OCI_INTER.	NTLM	Human	**ER**	ER filed — HTTP 407 — same root cause as T14
T16	SQL Dev Classic	JDBC THIN	OCI CLI file TOKEN_AUTH=OCI_TOKEN	NTLM	M2M	**SUCCESS**	Workaround for T14/T15 — file token bypasses proxy
T17	SQL Dev Classic	JDBC THIN	ojdbc-provider-oci seamless	NTLM+Basic	Human	**FAILURE**	ORA-18726 — same SDK proxy failure
T18	SQL Dev Classic	JDBC THIN	ojdbc-provider-oci seamless	No	Human	**SUCCESS**	Seamless OCI_INTERACTIVE works without proxy
T19	Java ojdbc17	JDBC THIN	OCI CLI manual file	No	M2M	**SUCCESS**	Baseline Java M2M — manual token
T20	Java ojdbc17	JDBC THIN	OCI CLI via ProcessBuilder	No	M2M	**SUCCESS**	Automated Java M2M — subprocess with proxy
T21	Java ojdbc17	JDBC THIN	OCI SDK GenerateScopedToken	NTLM	M2M	**WIP**	Open question — SDK proxy NTLM without credentials
T22	Java ojdbc17+UCP	UCP THIN	OCI SDK token Supplier	NTLM	M2M	**WIP**	Pool auto-refresh pattern — proxy open question
T23	Java ojdbc17	JDBC THICK	OCI SDK + Instant Client	NTLM	M2M	**WIP**	THICK + SDK in-process — proxy open question

5. Finding 1 — Python THIN Mode: Structural Failure at Oracle Net Connect Negotiation (T3 / T4)

Tests: T3 | T4 **SR:** Oracle Support SR — Open — Oracle Database Driver Engineering **Phase:** Connect negotiation (post-TLS, pre-authentication)

5.1 What Was Tested

python-oracledb 3.4.2 THIN mode — the default operating mode requiring no Oracle Instant Client, implementing the Oracle Net protocol in pure Python. Two separate token injection methods were exhaustively tested:

- T3 — Static 2-tuple: access_token=(token_string, private_key_string) passed via ConnectParams(), where token and private key are read from the OCI CLI-generated files (~/.oci/db-token/token + oci_db_key.pem). This is the documented primary method in Oracle Appendix B for OCI IAM in THIN mode.
- T4 — Callable: access_token=callable passed via the oci_tokens SDK plugin (extra_auth_params={'auth_type':'ConfigFileAuthentication'}), which returns the (token, private_key) tuple on demand. This is the in-process SDK equivalent of T3.

Both tests were run against the same ADB-S Private Endpoint with mTLS (ewallet.pem), same OCI IAM configuration, same user mapping. T1 and T2 (THICK mode, same endpoint, same token source) succeeded confirming the endpoint and IAM configuration are correct.

5.2 Observed Errors (both T3 and T4)

```
DPY-6005: cannot connect to database (CONNECTION_ID=...)
DPY-6000: Listener refused connection (similar to ORA-12506)

Failure phase: Oracle Net connect negotiation
TLS handshake: completed successfully
Token validity: confirmed (fresh token, valid JWT, valid private key)
```

5.3 Critical Technical Detail — The access_token Discrepancy

Oracle documentation (python-oracledb Appendix B — Thin and Thick Mode Differences) states:

```
"Open Authorization (OAuth 2.0) token based authentication connection strings and
 OCI IAM token based authentication connection strings are not supported.
 Use the access_token parameter of oracledb.ConnectParams() instead."

Source: https://python-oracledb.readthedocs.io/en/latest/user_guide/appendix_b.html
```

The documentation distinguishes between: (1) TOKEN_AUTH=OCI_TOKEN in the connect descriptor syntax — explicitly unsupported in THIN mode, and (2) access_token in ConnectParams() — documented as the supported alternative for THIN mode. T3 and T4 use exclusively the ConnectParams().access_token path. Both fail at connect negotiation. This means the failure is deeper than the token injection method — the THIN protocol state machine itself

does not handle the IAM challenge-response sequence that ADB-S Private Endpoint issues post-TLS.

5.4 Root Cause Analysis

After TLS handshake completion, ADB-S Private Endpoint sends an IAM authentication challenge as part of the Oracle Net protocol exchange. In THICK mode, libclntsh (the OCI native C library) handles this challenge natively, correctly constructing the PoP response. The pure-Python THIN protocol implementation in python-oracledb does not implement this IAM challenge-response sequence — the Python code does not recognise the challenge message type and fails at the listener negotiation layer, producing ORA-12506/DPY-6000. This is a protocol state machine incompatibility, not a token validity or configuration issue.

5.5 Validated Workaround

Use THICK mode with Oracle Instant Client 23.x. Invoke oracledb.init_oracle_client() before any connection. Use the oci_tokens SDK plugin for in-process token acquisition (T2) or file-based access_token tuple (T1):

```
import os, oracledb
import oracledb.plugins.oci_tokens  # registers OCI cloud-native auth plugin

# THICK mode — must call before any connect() or create_pool()
oracledb.init_oracle_client(lib_dir=r'[INSTANT_CLIENT_DIR]')

# ewallet.pem → THIN | cwallet.sso → THICK (wallet type differs by mode)
os.environ['TNS_ADMIN'] = r'[WALLET_DIR]'  # contains cwallet.sso + tnsnames.ora

# Option A: OCI SDK plugin (T2) — in-process token acquisition
conn = oracledb.connect(
    dsn='[TNS_ALIAS]',
    externalauth=True,
    extra_auth_params={'auth_type':'ConfigFileAuthentication', 'profile':'DEFAULT'}
)

# Option B: file-based token (T1) — token pre-generated by OCI CLI
token_file = open(r'[OCI_DB_TOKEN_DIR]/token').read().strip()
key_file   = open(r'[OCI_DB_TOKEN_DIR]/oci_db_key.pem').read().strip()
conn = oracledb.connect(
    dsn='[TNS_ALIAS]',
    access_token=(token_file, key_file)  # 2-tuple: (token, private_key)
)

# Verify IAM federation
with conn.cursor() as c:
    c.execute("SELECT sys_context('userenv','authenticated_identity'),"
              "       sys_context('userenv','authentication_method') FROM dual")
    print(c.fetchone())
```

6. Finding 2 — OCI_INTERACTIVE THICK Mode: Browser Launch Failure in libclntsh / oci.dll (T8 / T9 / T10 / T11)

Tests: T8 (SQLDev Classic, IC 23.26) | T9 (SQLDev Classic, IC 19.29) | T10 (SQL*Plus, IC 23.26) | T11 (SQL*Plus, IC 19.29)

SR: Oracle Support SR — Open — Oracle Client Engineering (studying oci.dll / libclntsh on Windows)

Versions tested: Oracle Instant Client 19.29.0.0.0 and 23.26.1.0.0, tested on SR Engineering request

6.1 What OCI_INTERACTIVE THICK Should Do

When TOKEN_AUTH=OCI_INTERACTIVE is set in sqlnet.ora or in the SECURITY section of a tnsnames.ora connect descriptor, Oracle Instant Client (libclntsh / oci.dll) is expected to:

- Parse the OCI_INTERACTIVE directive at connect time.
- Launch the system browser and initiate the OCI IAM OAuth2 authorization code flow.
- Receive the IAM callback, extract the db-token and oci_db_key.pem.
- Complete the Oracle Net connection using the token for PoP authentication — transparently, with no user intervention beyond the browser login.

This is the Oracle-native THICK implementation of Human SSO — entirely within libclntsh, independent of any Java SDK or ojdbc-extensions. Configuration format tested:

```
# sqlnet.ora
SQLNET.AUTHENTICATION_SERVICES = (OCI_IAM)

# tnsnames.ora
[TNS_ALIAS] =
  (DESCRIPTION =
    (ADDRESS = (PROTOCOL = TCPS)(HOST = [adb_host])(PORT = 1521))
    (CONNECT_DATA = (SERVICE_NAME = [service_name]))
    (SECURITY =
      (SSL_SERVER_DN_MATCH = YES)
      (TOKEN_AUTH = OCI_INTERACTIVE)
    )
  )

# Connect — no username, no password (slash = external auth)
sqlplus /@[TNS_ALIAS]
```

6.2 Two Distinct Failure Signatures Per Instant Client Version

Instant Client 19.29.0.0.0 — T9 (SQL Developer Classic THICK) and T11 (SQL*Plus)

```
ORA-01017: invalid credential or not authorized; logon denied
Failure phase: Authentication (post-negotiation)
Browser launched: NO
OCI IAM contacted: NO
```

OCI_INTERACTIVE does not exist in Instant Client 19.x. The library does not parse or act on TOKEN_AUTH=OCI_INTERACTIVE — it treats it as an unrecognised parameter and ignores it silently. With no credentials provided (slash syntax), the Oracle Net auth sequence falls through to standard password authentication. With no password provided, the result is ORA-01017. The error is generic and not informative — it does not indicate that the feature is simply absent.

Instant Client 23.26.1.0.0 — T8 (SQL Developer Classic THICK) and T10 (SQL*Plus)

```
ORA-25714: invalid value of PASSWORD_AUTH or TOKEN_AUTH in the client configuration
Failure phase: Client validation (before any network activity to OCI IAM)
Browser launched: NO
OCI IAM contacted: NO
```

Instant Client 23.x reads and processes TOKEN_AUTH=OCI_INTERACTIVE from sqlnet.ora and tnsnames.ora — the OCI_INTERACTIVE feature is present in this version. The library parses the parameter, identifies the intent, and reaches the browser-launch phase of the OCI_INTERACTIVE flow. However, at the client-side parameter validation stage, oci.dll raises ORA-25714 — the value OCI_INTERACTIVE is rejected as invalid in the current client configuration context. The browser never opens. No OCI IAM call is made. The failure is entirely client-side within the OCI native library.

6.3 The Diagnostic That Pins the Bug to libclntsh — Not SQL Developer

T8 (SQL Developer Classic, THICK IC 23.26) and T10 (SQL*Plus standalone, THICK IC 23.26) produce the identical ORA-25714 error with identical failure behaviour. The only difference between T8 and T10 is the client tool — SQL Developer vs SQL*Plus. SQL*Plus is a standalone CLI with no GUI layer, no JVM, no extension framework. Both use the same libclntsh / oci.dll. This definitively isolates the failure to the OCI native library itself, not to SQL Developer. Oracle Client Engineering is studying the oci.dll behaviour on Windows for this specific ADB-S Private Endpoint mTLS + OCI_INTERACTIVE scenario.

6.4 Contrast: OCI CLI Browser Flow Works (T12)

OCI CLI (T12) successfully performs the interactive browser flow and writes token + oci_db_key.pem to ~/.oci/db-token/. Browser opens, OCI IAM is contacted, token is returned. This confirms: OCI IAM is reachable, the machine has browser capability, the network allows the OAuth2 redirect. The failure in T8/T10 is entirely in the oci.dll OCI_INTERACTIVE implementation, not in OCI IAM, the network, or the ADB-S configuration.

7. Finding 3 — SQL Developer VSCode Extension: OCI_INTERACTIVE Provider Not Implemented (T7)

SR: Oracle Support SR — Open **Bug:** Oracle Engineering Code Bug — confirmed **Classification:** VSCODE SSO FOR DB USER AGAINST OCI DOES NOT WORK USING OCI_INTERACTIVE METHOD

7.1 Observed Error

```
ORA-18726: Failed to get a value from an OracleResourceProvider
Failure phase: AccessTokenProvider init — before any network activity
Proxy context: NTLM (note: failure is independent of proxy — see 7.2)
```

7.2 Root Cause

The ojdbc-extensions library (oracle/ojdbc-extensions on GitHub) provides the ojdbc-provider-oci module, which implements the OracleResourceProvider interface for OCI IAM. This provider uses the OCI Java SDK internally to perform the OAuth2 browser redirect and token acquisition. For it to work, the ojdbc-provider-oci JAR and its OCI Java SDK dependencies must be explicitly added to the JDBC driver classpath.

The SQL Developer VSCode Extension bundles ojdbc17 but does not bundle the ojdbc-provider-oci module or its OCI Java SDK dependencies. When oracle.jdbc.tokenAuthentication=OCI_INTERACTIVE is set, ojdbc17 looks up the OCI resource provider in the registered provider chain — the provider is not present — and raises ORA-18726 before any network activity. The failure phase is AccessTokenProvider init, which occurs before any proxy traversal is attempted. The NTLM proxy context in T7 is therefore irrelevant to this specific failure.

7.3 Contrast: SQL Developer Classic Works Without Proxy (T18)

SQL Developer Classic 24.3.1 with the full ojdbc-extensions dependency stack manually added to product.conf (ojdbc-provider-oci + oci-java-sdk-shaded-full + oci-java-sdk-identitydataplane — as documented in the Oracle A-Team blog by Ty Stahl, May 2025) correctly implements the OCI_INTERACTIVE provider chain. T18 (SQL Developer Classic, no proxy) succeeds: browser opens, OCI IAM OAuth2 flow completes, token returned, connection established. T7 (VSCode extension) fails because the extension does not include the equivalent library bundle.

8. Finding 4 — Corporate NTLM Proxy Blocks OCI_INTERACTIVE JDBC Flow: JVM HTTP Layer Failure (T14 / T15 / T17)

SRs: Two Oracle Support SRs — Closed **Bugs:** Two Enhancement Requests — official ER filed for SQL Developer Classic and SQL Developer VSCode

8.1 Architecture of the Failure

The OCI_INTERACTIVE JDBC flow via ojdbc-provider-oci (ojdbc-extensions) requires the JVM to make an outbound HTTPS call to OCI IAM OAuth2 endpoints (auth.*.oraclecloud.com) to initiate the browser redirect. The OCI Java SDK uses Apache HttpClient (via ApacheConnectorProvider) for this HTTP layer. In a corporate NTLM proxy environment:

- The corporate proxy requires NTLM authentication for CONNECT tunnel establishment.
- The JVM proxy configuration (passed via -Dhttps.proxyHost / -Dhttps.proxyPort system properties) provides host and port — but not NTLM credentials.
- The OCI Java SDK ApacheConnectorProvider logs: "Either username or password is null. Not configuring auth credentials for the proxy." — it does not attempt NTLM negotiation.
- The proxy returns HTTP 407 Authentication Required. The OCI IAM OAuth2 call fails before the browser can be launched.
- ojdbc-provider-oci receives the failure from the SDK, cannot obtain the token, and raises ORA-18726 to the JDBC caller.

8.2 Test Results

ID	Tool	Proxy	Auth Mode	HTTP Error	Result	Outcome / SR
T14	SQL Dev Classic	NTLM	OCI_INTERACTIVE	407	**ER**	Enhancement Request filed
T15	SQL Dev VSCode	NTLM	OCI_INTERACTIVE	407	**ER**	Enhancement Request filed
T17	SQL Dev Classic	NTLM+Basic	OCI_INTERACTIVE	407	**FAILURE**	Same SDK proxy failure
T16	SQL Dev Classic	NTLM	File OCI_TOKEN	None	**SUCCESS**	Workaround validated
T18	SQL Dev Classic	None	OCI_INTERACTIVE	None	**SUCCESS**	Without proxy: works
T12	OCI CLI	Proxy	CLI browser	None	**SUCCESS**	CLI proxy traversal works

8.3 Validated Workaround — File Token via OCI CLI Subprocess (T16)

The OCI CLI handles proxy traversal correctly via HTTPS_PROXY / HTTP_PROXY environment variables at the process level — independently of JVM system property proxy configuration. Generate the db-token externally via OCI CLI with proxy env vars, then configure SQL Developer to use the resulting file-based token (TOKEN_AUTH=OCI_TOKEN). This completely decouples Phase A (token acquisition, through the proxy) from Phase B (database connection, direct via VCN), bypassing the JVM NTLM limitation entirely:

```
# Step 1 — Token acquisition via OCI CLI (handles NTLM proxy)
set HTTPS_PROXY=http://[PROXY_HOST]:[PROXY_PORT]
oci iam db-token get
# Output: %USERPROFILE%\.oci\db-token\token + oci_db_key.pem (1h TTL)

# Step 2 — SQL Developer Classic Advanced Connection Properties
# oracle.jdbc.tokenAuthentication = OCI_TOKEN
# oracle.jdbc.tokenLocation       = C:\Users\[user]\.oci\db-token\
# Connection goes directly to ADB-S Private Endpoint via VCN — no proxy needed
```

9. Finding 5 — Java M2M Validated Pattern: ProcessBuilder Separation of Proxy Concerns (T19 / T20)

Result: SUCCESS — fully validated in production **Tests:** T19 (manual CLI token) | T20 (automated ProcessBuilder with proxy env injection)

9.1 The Architectural Principle

M2M Java token acquisition (Phase A) and JDBC connection (Phase B) have completely independent network paths:

- Phase A: OCI CLI → OCI IAM public endpoints. Requires HTTPS through the corporate proxy. Proxy injected via HTTPS_PROXY / HTTP_PROXY environment variables in the ProcessBuilder environment map — the subprocess inherits the proxy variables explicitly, independently of JVM system properties.
- Phase B: ojdbc17 → ADB-S Private Endpoint via OCI VCN fabric. No proxy traversal. The private endpoint is on the internal network. ojdbc17 reads token + oci_db_key.pem from the directory set via CONNECTION_PROPERTY_TOKEN_LOCATION, performs PoP signing internally.

The ProcessBuilder model makes this separation explicit and deterministic. It is the architectural key to M2M in any corporate proxy environment regardless of proxy type (NTLM, Basic, Kerberos).

9.2 Production Implementation — ojdbc17 23ai with ProcessBuilder (T20)

```
// ── Phase A: Token acquisition via OCI CLI subprocess ──
ProcessBuilder pb = new ProcessBuilder("oci", "iam", "db-token", "get");
Map<String, String> env = pb.environment();
// Proxy env vars injected explicitly into subprocess environment
env.put("HTTPS_PROXY", "http://[PROXY_HOST]:[PROXY_PORT]");
env.put("HTTP_PROXY",  "http://[PROXY_HOST]:[PROXY_PORT]");
pb.redirectErrorStream(true);
Process proc = pb.start();
boolean finished = proc.waitFor(60, TimeUnit.SECONDS);
int exitCode = proc.exitValue();
if (!finished || exitCode != 0) throw new RuntimeException("Token refresh failed: "
+ exitCode);

// Token freshness check (token TTL = 1h, refresh if older than 2 min)
Path tokenFile = Paths.get("[OCI_DB_TOKEN_DIR]", "token");
long ageSec = Duration.between(
    Files.getLastModifiedTime(tokenFile).toInstant(), Instant.now()).getSeconds();
if (ageSec > 120) log.warn("Token {}s old — may not be fresh", ageSec);

// ── Phase B: JDBC connection — no proxy, private endpoint via VCN ──
OracleDataSource ods = new OracleDataSource();
```

```
// TOKEN_AUTH=OCI_TOKEN can also be set in the TNS descriptor
ods.setURL("jdbc:oracle:thin:@[TNS_ALIAS]?TNS_ADMIN=[WALLET_DIR]");

Properties props = new Properties();
// ojdbc17 reads [TOKEN_DIR]/token + [TOKEN_DIR]/oci_db_key.pem automatically
// and performs PoP (Proof-of-Possession) signing before sending to ADB-S
props.setProperty(OracleConnection.CONNECTION_PROPERTY_TOKEN_AUTHENTICATION,
    "OCI_TOKEN");
props.setProperty(OracleConnection.CONNECTION_PROPERTY_TOKEN_LOCATION,
    "[OCI_DB_TOKEN_DIR]");
ods.setConnectionProperties(props);

try (Connection conn = ods.getConnection();
     Statement  st   = conn.createStatement();
     ResultSet  rs   = st.executeQuery(
         "SELECT sys_context('userenv','session_user'), "
       + "       sys_context('userenv','authenticated_identity'), "
       + "       sys_context('userenv','authentication_method') FROM dual")) {
    rs.next();
    log.info("session_user          : {}", rs.getString(1));
    log.info("authenticated_identity: {}", rs.getString(2)); // OCI IAM username
    log.info("authentication_method : {}", rs.getString(3)); // should be TOKEN
}
```

10. Work in Progress — Java OCI SDK In-Process Patterns (T21 / T22 / T23)

These three patterns eliminate the OCI CLI subprocess by calling OCI IAM directly from within the JVM via the OCI Java SDK (GenerateScopedAccessToken). They share a common open technical question: whether the OCI Java SDK can traverse a corporate NTLM proxy without explicit username + password credentials — a configuration common in Windows enterprise environments where NTLM is negotiated transparently at the OS level.

10.1 The Core Open Question — OCI Java SDK + NTLM Proxy Without Credentials

The OCI Java SDK uses Apache HttpClient (ApacheConnectorProvider) for HTTP calls to OCI IAM. Proxy is configured via JVM system properties (-Dhttps.proxyHost, -Dhttps.proxyPort). When proxyUser and proxyPassword are not provided, the SDK logs:

```
WARN  ApacheConnectorProvider: Either username or password is null.
      Not configuring auth credentials for the proxy.
```

In Windows enterprise environments, NTLM authentication is typically handled transparently by the OS (Windows Integrated Authentication) without requiring explicit credentials in application configuration. The OCI Java SDK + Apache HttpClient does not integrate with the Windows SSPI layer for NTLM credential negotiation. The implication: GenerateScopedAccessToken calls to OCI IAM fail at the proxy CONNECT tunnel with HTTP 407 — the same pattern observed in T14/T15/T17.

The T21/T22/T23 validation will determine whether: (a) configuring Apache HttpClient with explicit NTLM credentials solves the issue, or (b) an alternative HTTP connector (e.g., Jersey + NTLM authenticator, OkHttp, or native Java HttpClient with OS NTLM integration) must be used in the OCI SDK configuration for enterprise proxy compatibility.

10.2 T21 — JDBC THIN + OCI SDK In-Process (GenerateScopedAccessToken)

Eliminates the CLI subprocess. Uses CONNECTION_PROPERTY_TOKEN_AUTHENTICATION=OCI_API_KEY to instruct ojdbc17 to call the OCI Java SDK for token generation. ConfigFileAuthenticationDetailsProvider reads ~/.oci/config (DEFAULT profile) and calls IdentityDataPlaneClient.generateScopedAccessToken() in-process. This is the recommended pattern for containerized workloads (OCI Compute, OKE) where the CLI is not installed. Token caching via AccessToken.createJsonWebTokenCache(Supplier) avoids redundant IAM calls within the 1-hour TTL window.

10.3 T22 — UCP Pool + Token Supplier Pattern

Oracle Universal Connection Pool (UCP) supports a token supplier interface: OracleCommonDataSource.setTokenSupplier(Supplier<AccessToken>). The supplier is called by UCP whenever a new connection is created or when the pool expands after token expiry. The supplier calls GenerateScopedAccessToken, wraps the result in AccessToken.createJsonWebToken(token, privateKey), and returns it. UCP manages the connection lifecycle and token refresh transparently. This is the production-grade pattern for long-running Java application servers with connection pooling.

10.4 T23 — JDBC THICK + Instant Client + OCI SDK

JDBC THICK mode (jdbc:oracle:oci:@...) delegates the Oracle Net protocol stack and PoP signing to Oracle Instant Client (libclntsh / oci.dll), mirroring the architecture of T1/T2 in Python. The OCI Java SDK provides the token programmatically; libclntsh handles the PoP signing and TLS handshake natively. sqlnet.ora requires SQLNET.AUTHENTICATION_SERVICES=(OCI_IAM). This mode enables OCI native features (Oracle Advanced Security, NNE, event notification) in Java at the cost of requiring Oracle Instant Client installed on the application server.

11. Enterprise Pattern Selection Matrix

Deployment Context	Language	Pattern	Recommendation
M2M — no proxy	Python	THICK + oci_tokens	T2 — init_oracle_client() + extra_auth_params SDK plugin. In-process. No subprocess. IC 23.x required.
M2M — NTLM proxy	Python	THICK + oci_tokens	T2 — OCI SDK handles proxy via ~/.oci/config. THICK mode bypasses JVM proxy limitation entirely.
M2M — no proxy	Java	THIN + SDK in-process	T21 (WIP) — CONNECTION_PROPERTY_TOKEN_AUTHENTICATION=OCI_API_KEY. No subprocess. Cleanest pattern.
M2M — NTLM proxy	Java	THIN + CLI subprocess	T20 — ProcessBuilder + HTTPS_PROXY env vars. Validated in production. Separates proxy concerns.
M2M — pool/long-running	Java	UCP + token supplier	T22 (WIP) — setTokenSupplier(). Auto token refresh on pool expansion. Production app server pattern.
Human SSO — no proxy	Java	ojdbc-extensions OIDC	T18 — SQL Dev Classic + ojdbc-provider-oci (full lib stack). Browser opens, seamless login.
Human SSO — NTLM proxy	Any	CLI file token	T16 — OCI CLI with HTTPS_PROXY → file token. SQL Dev reads OCI_TOKEN. Bypasses JVM proxy.
AVOID	Python	THIN mode	T3/T4 FAILURE — structural protocol incompatibility on ADB-S Private Endpoint. SR open.
AVOID	Any	IC 19.x OCI_INTERACTIVE	T9/T11 — feature absent in 19.x. IC 23.x also fails (T8/T10 — ORA-25714 — SR open).
AVOID	Java	VSCode OCI_INTERACTIVE	T7 — Bug confirmed — provider not bundled in VSCode extension. Use Classic or file token.

12. Protocol Layer Failure Taxonomy

Each failure in this validation is precisely located within the Oracle Net protocol stack. The following taxonomy maps failures to their architectural layer:

Tests	Layer	Component	Error	Root Cause
T3/T4	Layer 4 — Oracle Net Negotiation	python-oracledb THIN protocol	DPY-6005/DPY-6000	THIN protocol state machine: IAM challenge not handled post-TLS
T8/T10	Layer 4 — Client Validation	libclntsh / oci.dll IC 23.26	ORA-25714	OCI_INTERACTIVE present but browser launch fails in oci.dll on Windows
T9/T11	Layer 4 — Client Validation	libclntsh / oci.dll IC 19.29	ORA-01017	OCI_INTERACTIVE feature absent in 19.x — silently ignored
T7	Layer 3 — Provider Resolution	ojdbc-provider-oci (VSCode)	ORA-18726	OCI provider not bundled in VSCode extension classpath
T14/T15	Layer 2 — Proxy Negotiation	OCI Java SDK ApacheConnector	ORA-18726 / 407	NTLM proxy: SDK does not configure auth credentials without user/pass
T17	Layer 2 — Proxy Negotiation	OCI Java SDK ApacheConnector	ORA-18726 / 407	Same as T14/T15 — NTLM+Basic proxy, same SDK limitation

Notably: no failure was observed at Layer 5 (PoP integrity), Layer 1 (TLS/mTLS), or at the ADB-S IAM configuration level. All validated failures are client-side, in the driver or connectivity library layer.

13. Conclusions

OCI IAM DB Token authentication for ADB-S is production-viable with deterministic results when the client stack and token acquisition mechanism are correctly matched to the deployment context. The 23 test scenarios in this validation demonstrate that the feature's behaviour is controlled by three independent variables: (1) the client protocol stack (THICK/THIN/JDBC), (2) the token acquisition mechanism (CLI/SDK/Interactive), and (3) the network topology (proxy type/direct). Mismatching any of these produces failures that are superficially similar but have completely different root causes.

- **Python-oracledb THIN mode —** structural incompatibility with ADB-S IAM at Oracle Net connect negotiation level. Documented as unsupported for connection string syntax; empirically confirmed also unsupported for access_token ConnectParams() injection on ADB-S Private Endpoint. SR open. Use THICK mode with IC 23.x.
- **OCI_INTERACTIVE THICK (libclntsh / oci.dll) —** feature absent in IC 19.x (ORA-01017 generic); feature present but broken in IC 23.x (ORA-25714 — browser never launched). Confirmed identical failure from SQL Developer and SQL*Plus — bug is in oci.dll, not in SQL Developer. SR open with Oracle Client Engineering.
- **OCI_INTERACTIVE JDBC via ojdbc-extensions —** works correctly without proxy (T18 SUCCESS). Fails under NTLM proxy (T14/T15/T17) due to OCI Java SDK ApacheConnectorProvider not configuring NTLM credentials. ERs filed. File token workaround validated (T16).
- **Java M2M ProcessBuilder pattern —** separation of proxy concerns is the architectural key. Token acquisition through the proxy (OCI CLI subprocess + HTTPS_PROXY env vars); JDBC connection direct to ADB-S Private Endpoint via VCN (no proxy). Validated in production.
- **Java OCI SDK in-process (T21/T22/T23) —** pending validation. Core open question: OCI Java SDK + NTLM proxy without explicit credentials — same SDK limitation observed in T14/T15/T17. Will determine viability of subprocess-free M2M patterns in corporate environments.

14. References

Oracle Docs — IAM Token Authentication — ADB-S (Enable + Configure). https://docs.oracle.com/en-us/iaas/autonomous-database/doc/iam-token-authentication.html

Oracle Docs — python-oracledb Authentication Methods (OCI IAM section). https://python-oracledb.readthedocs.io/en/stable/user_guide/authentication_methods.html

Oracle Docs — python-oracledb Appendix B — Thin vs Thick Differences. https://python-oracledb.readthedocs.io/en/latest/user_guide/appendix_b.html

Oracle Docs — python-oracledb ConnectParams — access_token parameter. https://python-oracledb.readthedocs.io/en/stable/api_manual/connect_params.html

Oracle Docs — python-oracledb Extending — oci_tokens plugin. https://python-oracledb.readthedocs.io/en/v3.4.0/user_guide/extending.html

Oracle Blog — Accessing ADB with IAM Token using Java (de Lavarene / McMahon). https://blogs.oracle.com/developers/accessing-autonomous-database-with-iam-token-using-java

Oracle Docs — JDBC Client-Side Security — Token-Based Authentication. https://docs.oracle.com/en/database/oracle/oracle-database/21/jjdbc/client-side-security.html

Oracle GitHub — oracle-db-examples — JDBC ConnectionSamples (JDBCDBTokenSample). https://github.com/oracle-samples/oracle-db-examples/tree/main/java/jdbc/ConnectionSamples

Oracle GitHub — ojdbc-extensions — ojdbc-provider-oci README. https://github.com/oracle/ojdbc-extensions/blob/main/ojdbc-provider-oci/README.md

Oracle Blog — A-Team: Seamless auth to Oracle DB with SQLDeveloper 23ai (Ty Stahl). https://www.ateam-oracle.com/seamless-authentication-to-the-oracle-database-with-sqldeveloper-23ai-jdbc-drivers-and-an-oci-identity-domain

Oracle Docs — OCI CLI — iam db-token get command reference. https://docs.oracle.com/en-us/iaas/tools/oci-cli/latest/oci_cli_docs/cmdref/iam/db-token/get.html

Oracle Docs — UCP — OracleCommonDataSource.setTokenSupplier(). https://docs.oracle.com/en/database/oracle/oracle-database/23/jjucp/

Oracle GitHub — OCI Java SDK — HttpProxyExample. https://github.com/oracle/oci-java-sdk/blob/master/bmc-examples/src/main/java/HttpProxyExample.java

Medium — python-oracledb 3.2 — Instance Principal Authentication (C. Jones). https://cjones-oracle.medium.com/python-oracledb-3-2-supports-instance-principal-authentication-71e390f2ca4e

Oracle Docs — Instant Client Downloads — supported versions. https://www.oracle.com/database/technologies/instant-client.html

Alessandro Moccia — Oracle ACE Program | Oracle Solution Architect | OCI Specialist

SR / Bug evidence in ADB_IAM_Token_Validation_Matrix.xlsx — github.com/alexmocciaoci/oracle-adb-iam-token-federation